Лабораторна робота №4

Фізико-інформовані нейромережі (PINN). Розв'язання початково-крайової задачі теплопровідності

Мета роботи

Набути практичних навичок застосування фізико-інформованих нейромереж (Physics-Informed Neural Networks, PINN) для розв'язання початково-крайової задачі теплопровідності; навчитися формулювати диференціальне рівняння та відповідні початкові й граничні умови, формувати функцію втрат PINN, навчати нейромережеву модель та оцінювати точність отриманого розв'язку шляхом порівняння з еталонним.

Теоретичний матеріал

Фізико-інформовані нейромережі (Physics-Informed Neural Networks, PINN) становлять клас методів, у яких апроксимація шуканого поля здійснюється штучною нейромережею, а фізичні закони залучаються безпосередньо через функцію втрат у вигляді резидуалів відповідних диференціальних рівнянь та умов задачі. На відміну від традиційних дискретизаційних схем, у PINN операторні похідні обчислюються засобами автоматичного диференціювання, що дає змогу обчислювати похідні необхідних порядків без явної скінченно-різницевої апроксимації диференціальних операторів. Водночас точність отриманого розв'язку залежить від апроксимаційної здатності нейромережі, вибору колокаційних точок та ефективності процедури оптимізації. Сучасний підхід до побудови фізико-інформованих нейромереж був систематизований і набув широкого поширення після роботи M. Raissi та співавторів, опублікованої у 2019 році. Відтоді PINN активно застосовуються для прямих та обернених задач математичної фізики, у яких фізичні закономірності задаються диференціальними рівняннями.

Математична модель теплопровідності базується на законі Фур'є $\mathbf{q}=-k\nabla T$ та локальному балансі енергії. Нехай $\Omega\subset R^d$ є просторовою областю з гладкою межею $\partial\Omega$, $t\in(0,T]$— часовий інтервал, $T(\boldsymbol{x},t)$ — температура, $k>0$ — коефіцієнт теплопровідності, $\rho>0$— густина, $c>0$ — питома теплоємність, $Q(\boldsymbol{x},t)$— об'ємне джерело теплоти. Тоді нестаціонарна задача формулюється у вигляді параболічного рівняння

$$\rho c\,\frac{\partial T}{\partial t} = \nabla\cdot\big(k\nabla T\big) + Q(\mathbf{x},t) \quad \text{в } \Omega\times(0,T],$$

доповненого граничними та початковою умовами. Типи граничних умов задаються у стандартній формі: умови першого роду (умови Діріхле) $T=T_D$ на частині межі $\Gamma_D\subseteq\partial\Omega$; умови другого роду (умови Неймана) $-k\nabla T\cdot\boldsymbol{n}=q_N$ на $\Gamma_N\subseteq\partial\Omega$; умови третього роду (конвективні умови Робіна $-k\nabla T\cdot\boldsymbol{n}=h\,(T-T_C)$ на $\Gamma_R\subseteq\partial\Omega$, де $\boldsymbol{n}$ — зовнішня нормаль, $h>=0$ — коефіцієнт тепловіддачі, T_C— температура навколишнього середовища. Для $t=0$ специфікується початковий стан $T(\boldsymbol{x},0)=T_0(\boldsymbol{x})$. У випадку просторової однорідності $k=$const та відсутності джерел $Q\equiv0$ рівняння редукується до канонічної форми $\partial_t T-\alpha\Delta T=0$ з температуропровідністю $\alpha=k/(\rho c)$; у стаціонарному режимі, відповідно, маємо еліптичну задачу $-\alpha\Delta T=0$.

В інтересах чисельної стабільності та коректного масштабування параметрів доцільно виконати безрозмірізацію. Нехай L позначає характерний лінійний масштаб області, а $\tau=L_2/\alpha$ — характерний дифузійний час, тоді введення змінних $\boldsymbol{x}^*=\boldsymbol{x}/L$ і $t^*=t/\tau$ приводить у випадку $Q\equiv0$ до безрозмірної параболічної моделі $\partial_{t^*}T^*-\Delta^*T^*=0$. На межах

із конвекцією природно виникає число Біо Bi=hL/k, а часову еволюцію зручно параметризувати числом Фур'є $F_o=\alpha t/L_2$. На практичному рівні для навчання PINN координати й час афінно переносять до компактних інтервалів на кшталт [−1,1] або [0,1], тоді як температуру центровано нормують відносно еталонного рівня T_{ref}, що забезпечує порядок одиниці для динамічного діапазону цільової змінної.

У межах підходу PINN розв'язок апроксимується нейромережею T_θ: ($\boldsymbol{x}$,t)↦**R**, де θ позначає сукупність параметрів моделі. Функціонал втрат будується як зважена сума складників, що відповідають залишкам фізичного рівняння, граничним і початковій умовам, а за наявності вимірювань — також даним. Для однорідної задачі без джерел природною є така конструкція:

$$\mathcal{L}(\theta) = \lambda_f \frac{1}{N_f}\sum_{i=1}^{N_f}\left(\partial_t \hat{T}_\theta(\mathbf{x}_i,t_i) - \alpha\,\Delta\hat{T}_\theta(\mathbf{x}_i,t_i)\right)^2 + \lambda_b \frac{1}{N_b}\sum_{j=1}^{N_b}\mathcal{B}(\hat{T}_\theta;\mathbf{x}_j,t_j)^2 + \lambda_0 \frac{1}{N_0}\sum_{k=1}^{N_0}\left(\hat{T}_\theta(\mathbf{x}_k,0) - T_0(\mathbf{x}_k)\right)^2.$$

Оператор B визначається відповідно до типу граничної умови: для Діріхле $B=T_\theta-T_D$; для Неймана $B=-k\nabla T_\theta\cdot\boldsymbol{n}-q_N$; для Робіна $B=-k\nabla T_\theta\cdot\boldsymbol{n}-h(T_\theta-T_C)$. Похідні за часом і просторовими координатами, необхідні для обчислення резидуалу диференціального рівняння, визначаються засобами автоматичного диференціювання у межах тієї самої обчислювальної граф-схеми, що й вихід нейромережі. Це усуває необхідність явно апроксимувати похідні різницевими формулами.

Задання умов задачі може здійснюватися двома комплементарними підходами. У «м'якій» постановці умови імплементуються через штрафні доданки у функціоналі втрат L із належним вибором ваг λ_b та λ_0. У «жорсткій» постановці використовується конструкція $T_\theta(\mathbf{x},t) = g(\mathbf{x},t) + \phi(\mathbf{x},t)\,N_\theta(\mathbf{x},t)$, де g є відомою функцією, що точно задовольняє відповідні умови, а ϕ— згладжувальна функція, яка занулюється на відповідних підмножинах (зокрема на $\partial\Omega$ для умов Діріхле чи на t=0 для початкової умови). Така конструкція забезпечує точне дотримання умов і зменшує залежність від вибору ваг штрафів, хоча її побудова у складних геометріях може бути нетривіальною.

Ефективність навчання істотно залежить від вибору колокаційних точок. Усередині області $\Omega\times(0,T]$ застосовуються рівномірні або квазірівномірні схеми семплювання, зокрема латинський гіперкуб, тоді як на межі $\partial\Omega$ точки розміщують рівномірно вздовж кожного сегмента з урахуванням його міри. У присутності граничних шарів чи різких градієнтів температури корисним є нерівномірне семплювання з підвищеною щільністю у критичних зонах. Регулярна перегенерація батчів запобігає надмірному пристосуванню до фіксованого набору точок і покращує узагальнювальну здатність.

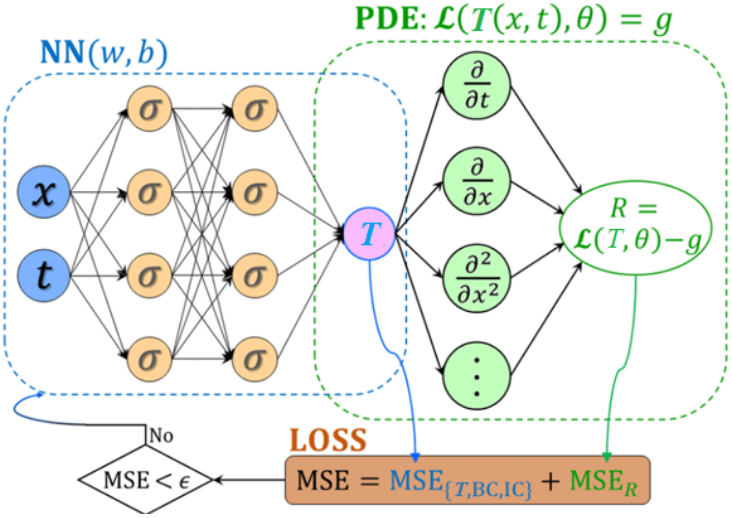


Рис.1. Загальна структура фізико-інформованої нейромережі PINN

Архітектурно модель найчастіше реалізується як багатошаровий персептрон із гладкими активаціями, зокрема *tanh* або синусоїдальними, що забезпечують належні властивості похідних другого порядку, необхідних для Лапласіана. Вхідними ознаками є просторово-часові координати (x,t) для 2D/3D — (x,y,t) або (x,y,z,t), за потреби попередньо безрозмірні. На виході отримуємо наближення температури $T_\theta(\boldsymbol{x},t)$. Ініціалізація ваг за схемами *Xavier* або, для синусоїдальних активацій, спеціалізованими SIREN-параметрами зменшує ризики деградації градієнтів. В межах обчислювального графа, засобами автоматичного диференціювання обчислюються похідні $\partial_t T, \nabla T, \Delta T$. Оптимізаційна процедура зазвичай поєднує стохастичний градієнтний метод типу *Adam* на початковій фазі з подальшим детермінованим доопрацюванням за L-BFGS, що демонструє ефективність для гладких функцій втрат. Баланс складників функціонала контролюється ваговими коефіцієнтами $\lambda_f,\lambda_b,\lambda_0$; відхилення граничних умов вимагає або їх підсилення, або переходу до жорсткого задання. Додаткові прийоми стабілізації включають обрізання градієнтів та нормування входів.

Валідація результатів здійснюється шляхом порівняння з еталонними розв'язками, які, залежно від постановки, можуть бути або аналітичними, або високоточними чисельними (наприклад, скінченно-різницевими чи скінченно-елементними) на згущених сітках. Узагальненою інтегральною мірою точності слугує відносна норма L_2,

$$\varepsilon_{L^2} = \frac{\|\hat{T} - T^\star\|_2}{\|T^\star\|_2},$$

де $T^\star$ позначає еталонний розв'язок; додатково аналізують показники RMSE та MAE, а також середні порушення граничних і початкових умов. Для стаціонарних задач корисною є перевірка інтегральних балансів теплового потоку; для задач із конвективним теплообміном — узгодження макропараметрів із числом Біо.

Окремий інтерес становлять інверсні постановки, у яких невідомі параметри середовища, зокрема просторово-залежний коефіцієнт $k(\boldsymbol{x})$ або зовнішнє джерело $Q(\boldsymbol{x},t)$), оцінюються одночасно з відновленням температурного поля. У цих випадках параметри включають до вектора θ або параметризують окремими підмережами; функціонал доповнюється узгодженням із даними вимірювань. Такі задачі потребують регуляризації на параметричних полях (наприклад, L_2 або TV-регуляризації), ретельного масштабування та коректного вибору вагових коефіцієнтів у функціоналі.

Приклад

Для демонстраційних цілей у лабораторній роботі розглянемо одномірну нестаціонарну задачу в стрижні $\boldsymbol{x}\in(0,L)$ з умовами Діріхле на кінцях та заданим початковим профілем температури, тобто $\partial_t T = \alpha\,\partial_{xx} T,\ T(0,t) = T_L,\ T(L,t) = T_R,\ T(x,0) = T_0(x)$. За фіксованих значень $T_L=T_R=0$ та достатньої гладкості $T_0=sin(\pi x)$розв'язок подається у вигляді $T^* = e^{-\pi^2 \alpha t}\sin(\pi x)$ ($\alpha = 1$), що забезпечує зручний еталон для кількісної оцінки похибок моделі PINN. За відсутності аналітичного розв'язку еталон формують за допомогою класичної чисельної схеми з доведеними властивостями збіжності.

У коді генеруються колокаційні точки, за допомогою автоматичного диференціювання обчислюються залишки рівняння, граничних та початкової умов, після чого мережа навчається методом Adam, а точність розв'язку оцінюється за метрикою похибки L_2.

```
      # TensorFlow 2.x PINN: 1D heat equation u_t - alpha u_xx = 0
      # Domain: x in (0,1), t in (0,1]; BC: u(0,t)=0, u(1,t)=0; IC:
u(x,0)=sin(pi x)
      # Analytic solution (for validation): u*(x,t) = exp(-pi^2 * alpha * t)
* sin(pi * x)
```

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

tf.random.set_seed(42)
np.random.seed(42)

# ------------------------------
# Problem settings
# ------------------------------
alpha = 1.0
xmin, xmax = 0.0, 1.0
tmin, tmax = 0.0, 1.0

def u_ic(x):
    return tf.sin(np.pi * x)

def u_exact(x, t):
    return tf.exp(- (np.pi**2) * alpha * t) * tf.sin(np.pi * x)

# ------------------------------
# Sampling of collocation points
# ------------------------------
N_f = 10_000   # interior (PDE) points
N_b = 1_000    # boundary points (both ends combined)
N_0 = 1_000    # initial-condition points (t=0)

# PDE points: x in (0,1), t in (0,1]
x_f = tf.random.uniform((N_f, 1), xmin, xmax)
t_f = tf.random.uniform((N_f, 1), tmin, tmax)

# Boundary points: left (x=0) and right (x=1), t in (0,1]
t_b = tf.random.uniform((N_b, 1), tmin, tmax)
x_bL = tf.zeros_like(t_b) + xmin
x_bR = tf.zeros_like(t_b) + xmax

# Initial-condition points: x in (0,1), t=0
x_0 = tf.random.uniform((N_0, 1), xmin, xmax)
t_0 = tf.zeros_like(x_0) + tmin
u_0 = u_ic(x_0)

# Stack helper for model input
def to_input(x, t):
    return tf.concat([x, t], axis=1)  # shape (N,2)

# ------------------------------
# PINN model
# ------------------------------
def build_model(width=128, depth=5):
    model = tf.keras.Sequential()
    model.add(tf.keras.layers.InputLayer(input_shape=(2,)))
    for _ in range(depth):
        model.add(tf.keras.layers.Dense(width, activation=tf.nn.tanh,
kernel_initializer=tf.keras.initializers.GlorotNormal()))
    model.add(tf.keras.layers.Dense(1, activation=None))
    return model

model = build_model(width=128, depth=6)
```

```
        # ----------------------------
        # AutoDiff: u_t, u_x, u_xx
        # ----------------------------
        @tf.function
        def pde_residual(x, t):
            """Compute PDE residual R = u_t - alpha*u_xx at given (x,t)."""
            with tf.GradientTape(persistent=True) as tape2:
                tape2.watch([x, t])
                with tf.GradientTape(persistent=True) as tape1:
                    tape1.watch([x, t])
                    u = model(to_input(x, t))  # (N,1)
                u_x = tape1.gradient(u, x)
                u_t = tape1.gradient(u, t)
            u_xx = tape2.gradient(u_x, x)
            del tape1, tape2
            return u_t - alpha * u_xx, u  # residual and u

        # ----------------------------
        # Loss components
        # ----------------------------
        mse = tf.keras.losses.MeanSquaredError()

        @tf.function
        def loss_and_grads():
            with tf.GradientTape() as tape:
                # PDE residual loss
                R_f, _ = pde_residual(x_f, t_f)
                loss_pde = tf.reduce_mean(tf.square(R_f))

                # BC losses (Dirichlet u=0 at x=0 and x=1)
                _, u_bL = pde_residual(x_bL, t_b)  # only u used
                _, u_bR = pde_residual(x_bR, t_b)
                loss_bc       =       tf.reduce_mean(tf.square(u_bL))       +
tf.reduce_mean(tf.square(u_bR))

                # IC loss (t=0: u(x,0)=sin(pi x))
                _, u_init = pde_residual(x_0, t_0)
                loss_ic = tf.reduce_mean(tf.square(u_init - u_0))

                # total loss (weights can be tuned if needed)
                loss = loss_pde + loss_bc + loss_ic
            grads = tape.gradient(loss, model.trainable_variables)
            return loss, loss_pde, loss_bc, loss_ic, grads

        optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)

        # ----------------------------
        # Training loop (Adam)
        # ----------------------------
        EPOCHS = 5_000
        log_every = 100

        history = {"loss": [], "pde": [], "bc": [], "ic": []}

        for ep in range(1, EPOCHS + 1):
            loss, lp, lb, li, grads = loss_and_grads()
            optimizer.apply_gradients(zip(grads, model.trainable_variables))

            if ep % log_every == 0 or ep == 1:
                history["loss"].append(float(loss))
                history["pde"].append(float(lp))
```

```
            history["bc"].append(float(lb))
            history["ic"].append(float(li))
            print(f"Epoch  {ep:5d}:  total={loss:.3e}  |  pde={lp:.3e}  |
bc={lb:.3e} | ic={li:.3e}")

    # ----------------------------
    # Validation on a grid & metrics
    # ----------------------------
    nx, nt = 200, 200
    xg = np.linspace(xmin, xmax, nx).reshape(-1,1).astype(np.float32)
    tg = np.linspace(tmin, tmax, nt).reshape(-1,1).astype(np.float32)
    Xg, Tg = np.meshgrid(xg, tg)  # shape (nt,nx)

    x_flat = tf.constant(Xg.reshape(-1,1))
    t_flat = tf.constant(Tg.reshape(-1,1))

    u_pred = model(to_input(x_flat, t_flat))
    u_pred = tf.reshape(u_pred, (nt, nx))

    u_ref = u_exact(tf.constant(Xg), tf.constant(Tg))

    # Relative L2 error
    num = tf.norm(u_pred - u_ref)
    den = tf.norm(u_ref)
    rel_l2 = (num / den).numpy()
    print(f"\nRelative L2 error on grid: {rel_l2:.3e}")

    # ----------------------------
    # Visualizations
    # ----------------------------
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1)
    plt.imshow(u_ref.numpy(), origin='lower', aspect='auto',
               extent=[xmin, xmax, tmin, tmax])
    plt.title("Exact u*(x,t)")
    plt.xlabel("x"); plt.ylabel("t"); plt.colorbar()

    plt.subplot(1,3,2)
    plt.imshow(u_pred.numpy(), origin='lower', aspect='auto',
               extent=[xmin, xmax, tmin, tmax])
    plt.title("PINN û(x,t)")
    plt.xlabel("x"); plt.ylabel("t"); plt.colorbar()

    plt.subplot(1,3,3)
    plt.imshow((u_pred-u_ref).numpy(), origin='lower', aspect='auto',
               extent=[xmin, xmax, tmin, tmax])
    plt.title("Error (PINN - exact)")
    plt.xlabel("x"); plt.ylabel("t"); plt.colorbar()
    plt.tight_layout()
    plt.show()

    # 1D profiles at selected times
    for t_sel in [0.0, 0.25, 0.5, 1.0]:
        x_line = tf.linspace(xmin, xmax, 200)[:,None]
        t_line = tf.ones_like(x_line) * t_sel
        u_p = model(to_input(x_line, t_line))
        u_e = u_exact(x_line, t_line)
        plt.plot(x_line, u_e, ls='--', label=f'Exact t={t_sel}')
        plt.plot(x_line, u_p, label=f'PINN t={t_sel}')
    plt.legend(); plt.xlabel("x"); plt.ylabel("u")
    plt.title("Profiles at selected times")
```

```
plt.show()
```

```
Epoch     1: total=4.743e-01 | pde=2.115e-04 | bc=1.237e-03 | ic=4.729e-01
Epoch   100: total=1.646e-01 | pde=8.819e-03 | bc=6.207e-02 | ic=9.373e-02
Epoch   200: total=1.940e-02 | pde=7.036e-03 | bc=5.243e-03 | ic=7.125e-03
Epoch   300: total=4.417e-03 | pde=2.433e-03 | bc=1.150e-03 | ic=8.341e-04
Epoch   400: total=1.500e-02 | pde=1.232e-02 | bc=2.118e-03 | ic=5.623e-04
Epoch   500: total=1.143e-03 | pde=6.214e-04 | bc=3.735e-04 | ic=1.480e-04
Epoch   600: total=1.511e-02 | pde=6.104e-03 | bc=7.796e-03 | ic=1.209e-03
Epoch   700: total=6.958e-04 | pde=3.664e-04 | bc=2.448e-04 | ic=8.462e-05
Epoch   800: total=7.696e-04 | pde=4.123e-04 | bc=2.584e-04 | ic=9.885e-05
Epoch   900: total=2.294e-03 | pde=1.920e-03 | bc=3.103e-04 | ic=6.374e-05
Epoch  1000: total=3.755e-03 | pde=2.719e-03 | bc=9.494e-04 | ic=8.590e-05
Epoch  1100: total=2.687e-03 | pde=2.371e-03 | bc=2.467e-04 | ic=6.973e-05
Epoch  1200: total=4.339e-03 | pde=3.396e-03 | bc=8.562e-04 | ic=8.605e-05
Epoch  1300: total=2.709e-03 | pde=1.307e-03 | bc=2.284e-04 | ic=1.173e-03
Epoch  1400: total=1.521e-03 | pde=1.313e-03 | bc=1.623e-04 | ic=4.531e-05
Epoch  1500: total=6.603e-04 | pde=2.748e-04 | bc=2.511e-04 | ic=1.343e-04
Epoch  1600: total=2.311e-03 | pde=1.919e-03 | bc=1.768e-04 | ic=2.159e-04
Epoch  1700: total=1.834e-03 | pde=1.535e-03 | bc=2.312e-04 | ic=6.834e-05
Epoch  1800: total=1.482e-03 | pde=1.310e-03 | bc=9.499e-05 | ic=7.707e-05
Epoch  1900: total=1.489e-03 | pde=1.176e-03 | bc=4.944e-05 | ic=2.631e-04
Epoch  2000: total=9.312e-03 | pde=7.059e-03 | bc=2.166e-03 | ic=8.657e-05
Epoch  2100: total=6.077e-03 | pde=4.125e-03 | bc=5.801e-04 | ic=1.371e-03
Epoch  2200: total=9.688e-04 | pde=3.591e-04 | bc=4.283e-04 | ic=1.813e-04
Epoch  2300: total=6.919e-04 | pde=1.587e-04 | bc=1.541e-04 | ic=3.791e-04
Epoch  2400: total=7.087e-04 | pde=3.010e-04 | bc=2.572e-04 | ic=1.505e-04
Epoch  2500: total=3.434e-03 | pde=1.087e-03 | bc=1.071e-03 | ic=1.275e-03
Epoch  2600: total=1.975e-03 | pde=1.725e-03 | bc=1.936e-04 | ic=5.656e-05
Epoch  2700: total=1.060e-03 | pde=3.008e-04 | bc=5.017e-04 | ic=2.577e-04
Epoch  2800: total=5.419e-05 | pde=2.931e-05 | bc=1.905e-05 | ic=5.827e-06
Epoch  2900: total=3.040e-04 | pde=2.575e-04 | bc=3.522e-05 | ic=1.126e-05
Epoch  3000: total=1.798e-03 | pde=9.725e-04 | bc=3.314e-04 | ic=4.940e-04
Epoch  3100: total=1.450e-03 | pde=6.520e-04 | bc=2.578e-04 | ic=5.399e-04
Epoch  3200: total=4.489e-04 | pde=4.241e-04 | bc=1.482e-05 | ic=9.931e-06
Epoch  3300: total=6.986e-04 | pde=4.710e-04 | bc=1.497e-04 | ic=7.784e-05
Epoch  3400: total=1.165e-03 | pde=1.662e-04 | bc=4.378e-04 | ic=5.615e-04
Epoch  3500: total=2.324e-04 | pde=1.975e-04 | bc=2.896e-05 | ic=5.947e-06
Epoch  3600: total=2.790e-04 | pde=2.262e-04 | bc=3.485e-05 | ic=1.802e-05
Epoch  3700: total=2.250e-04 | pde=1.348e-04 | bc=1.465e-05 | ic=7.557e-05
Epoch  3800: total=2.546e-03 | pde=9.523e-04 | bc=9.193e-04 | ic=6.745e-04
Epoch  3900: total=1.666e-04 | pde=1.228e-04 | bc=3.363e-05 | ic=1.018e-05
Epoch  4000: total=7.987e-03 | pde=6.353e-03 | bc=5.251e-04 | ic=1.108e-03
Epoch  4100: total=1.703e-04 | pde=8.098e-05 | bc=5.067e-05 | ic=3.865e-05
Epoch  4200: total=8.504e-04 | pde=7.192e-04 | bc=8.043e-05 | ic=5.072e-05
Epoch  4300: total=1.796e-03 | pde=1.563e-03 | bc=1.584e-04 | ic=7.468e-05
Epoch  4400: total=4.352e-04 | pde=4.004e-04 | bc=1.140e-05 | ic=2.342e-05
Epoch  4500: total=1.415e-03 | pde=1.196e-03 | bc=1.772e-04 | ic=4.214e-05
Epoch  4600: total=4.266e-03 | pde=2.956e-03 | bc=3.054e-04 | ic=1.005e-03
Epoch  4700: total=3.366e-04 | pde=2.799e-04 | bc=2.691e-05 | ic=2.982e-05
Epoch  4800: total=7.623e-04 | pde=6.831e-04 | bc=4.669e-05 | ic=3.258e-05
Epoch  4900: total=2.016e-04 | pde=1.290e-04 | bc=1.051e-05 | ic=6.211e-05
Epoch  5000: total=1.548e-03 | pde=1.412e-03 | bc=6.892e-05 | ic=6.676e-05
```

Рис.2. Збіжність навчання

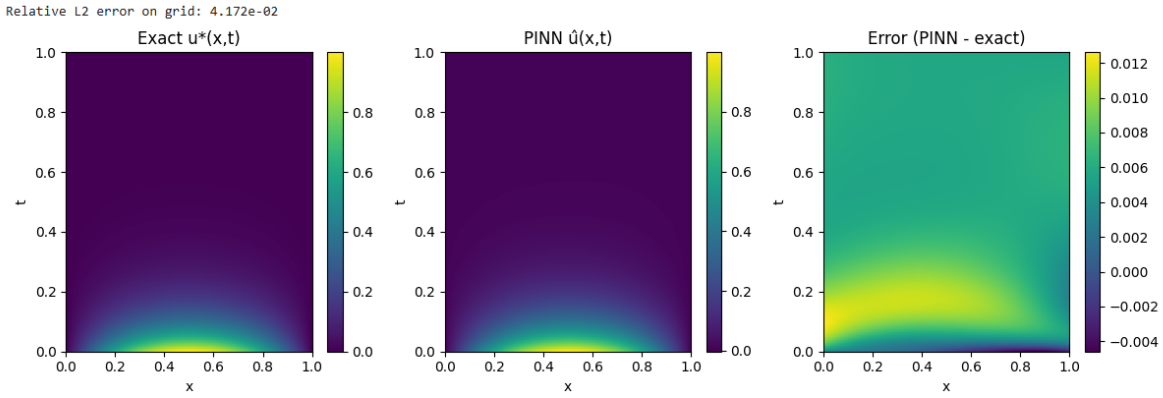


Рис.3. Теплові карти

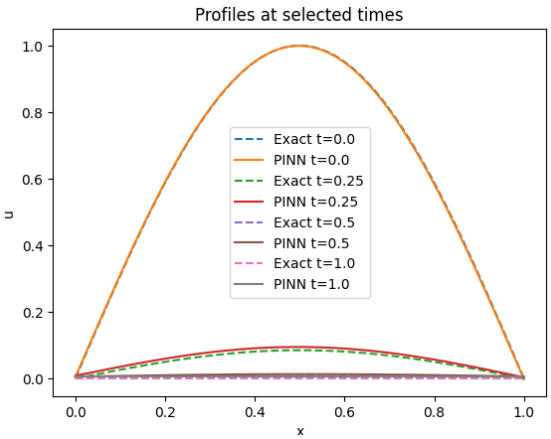


Рис.4. Профілі в часі

У процесі навчання спостерігається загальна тенденція до зменшення функції втрат та її складників, хоча на окремих етапах виникають локальні коливання, характерні для оптимізації методом Adam. До 5 000 ітерацій усі терміни опускаються до 10^{-3} -10^{-5} . Розрізи за t={0, 0.25, 0.5, 1.0} демонструють майже повне накладання кривих PINN і еталона; граничні значення на x=0, x=1 виконуються. Мережа коректно відтворює динаміку 1D теплопровідності; основна похибка сконцентрована на ранніх часах.

Індивідуальні завдання

Розв'язання крайової задачі теплопровідності за допомогою PINN та дослідження впливу гіперпараметрів.

1. Підготовка постановки

− Визначити геометрію Ω, часовий інтервал [0,T], типи граничних умов (Діріхле/Нейман/Робін) та початкову умову.

− Виконати безрозміризацію.

− За потреби задати функцію джерела із чітким аналітичним записом.

2. Формування колокаційних множин

− Згенерувати точки всередині області Ω.

− Згенерувати точки на межі ∂Ω.

− Для нестаціонарних задач підготувати точки для початкової умови.

− Масштабувати вхідні змінні до [0,1] або [−1,1]; зафіксувати seed і стратегію семплінгу (рівномірний/LHS/importance).

3. Модель PINN і функція втрат

− Побудувати MLP із гладкими активаціями (`tanh` або `sin`) і ініціалізацією Xavier/SIREN.

− Реалізувати AutoDiff для обчислення похідних.

− Задати функцію втрат.

4. Навчання і дослідження впливу параметрів

− Провести 2–3 серії тренувань, варіюючи: архітектуру (глибина/ширина), активації (`tanh` vs `sin`), ваги λ_f, λ_b, λ_0, стратегію семплінгу.

− Режим оптимізації: Adam (lr $10^{-3}\rightarrow10^{-4}$ → L-BFGS (за можливості). Раннє зупинення за валідаційною сіткою.

− Окремо простежити баланс складників втрат у часі.

5. Оцінювання

− Порівняти вихід мережі з еталоном: відносна L2-похибка, RMSE/MAE на щільній сітці.

− Перевірити виконання ГУ/ПУ в нормі L2(∂Ω)/ L2(Ω).

− Для задач із джерелом — перевірити тепловий баланс:

$$\frac{d}{dt}\int_{\Omega} u\, d\Omega = \int_{\Omega} s\, d\Omega - \int_{\partial\Omega} \partial_n u\, d\Gamma.$$

−Для інверсії параметрів — оцінити похибки відновлення параметрів.

6. Висновки

- − Побудувати карти поля u(x,t), карти похибок, вихід мережі по характерних перерізах/часах.
- − Показати еволюцію складників функції втрат, графіки залежності похибки L2 від вибраних гіперпараметрів (напр., λ_b, глибина, тип активації).
- − Сформувати висновки: як кожен параметр вплинув на точність, виконання умов, стабільність оптимізації та необхідну кількість колокаційних точок..

Типові налаштування: MLP 6×128, `tanh`; λ_f=1, $\lambda_b \in [5,50]$, $\lambda_0 \in [1,10$; Adam $10^{-3} \to 10^{-4}$, 8–12k кроків; опційно L-BFGS.

Еталони: аналітичний або FDM/FEM на дрібній сітці.

1) $u(x,0) = 2x(x+1) + 0{,}5$; $u(0,t) = 2t + 0{,}5$; $u(1,t) = 2t + 4{,}5$;

Додатково:

1. Порівняти Adam проти Adam→L-BFGS.
2. Дослідити вплив $\lambda_b \in \{5,10,20\}\}$ на порушення ГУ.
3. Обчислити тепловий потік на межах у 3 моментах часу.

2) $u(x,0) = \sin(x + 0{,}5)$; $u(0,t) = 2t + 0{,}4794$; $u(1,t) = t + 0{,}9975$;

Додатково:

1. Зробити curriculum у часі (спершу $t \le 0.4$), порівняти точність.
2. `tanh` vs `sin` (SIREN) — порівняти L_2.

3) $u(x,0) = (x^2 + 1{,}5)\sin \pi x$; $u(0,t) = 16t$; $u(1,t) = (t+2)t$;

Додатково:

1. Importance-sampling у зонах найбільших градієнтів (оцінити виграш).
2. Перевірити стабільність по 3 seed; подати середні/СКО метрик.

4) $u(x,0) = (x^2 + 1)\sin \pi x$; $u(0,t) = 4t$; $u(1,t) = 10t$;

Додатково:

1. Збільшити глибину з 6×128 до 8×200 — вплив на збіжність.
2. Додати слабкий Робін на $x = 1$: $-u_x = \mathrm{Bi}(u - u_\infty)$ з $\mathrm{Bi} = 1$, $u_\infty = u(1,t)$ базового кейсу; оцінити зміну потоку.

5) $u(x,0) = \cos 2x$; $u(0,t) = 1 - 2t$; $u(1,t) = t - 0{,}4161$;

Додатково:

1. Ввести мале $s(x,t) = 0.5\, e^{-80(x-0.5)^2} \mathbf{1}_{t>0.4}$; показати вплив на профілі.
2. Порівняти рівномірний семплінг vs LHS.

6) $u(x,0) = x(x+1)$; $u(0,t) = 3t$; $u(1,t) = 2(t+1)$;

Додатково:

1. Перевірити тепловий баланс на [0,1] (для s≡0 баланс зводиться до потоку на межах).
2. Дослідити вплив $N_f \in \{5\text{k},10\text{k},20\text{k}\}$ на точність.

7) $u(x,0) = \lg(x+1) + 1{,}3$; $u(0,t) = t^2 + 1{,}3$; $u(1,t) = t^2 + 1{,}6010$;

Додатково:

1. Дослідити `tanh` vs Swish — збіжність/точність.
2. Перевірити стабільність по 4 seed; подати середні/СКО метрик.

8) $u(x,0) = \sin 2x$; $u(0,t) = 2t$; $u(1,t) = 0{,}9093$;

Додатково:

1. Змінити масштаби входів: [0,1]vs [−1,1]; порівняти.
2. Побудувати часові серії $u(0.25,t),u(0.5,t),u(0.75,t)$.

9) $u(x,0) = 3x(2-x)$; $u(0,t) = t$; $u(1,t) = t+3$;

Додатково:

1. Додати шум 1% до ПУ; показати робастність (3 запуски).
2. Порівняти L_2 з/без ранньої зупинки.

10) $u(x,0) = (x^2+1)\sin \pi x$; $u(0,t) = 16t$; $u(1,t) = t^2$;

Додатково:

1. Порівняти Adam проти Adam→L-BFGS.
2. Дослідити вплив $\lambda_b \in \{5,10,20\}\}$ на порушення ГУ.
3. Порівняти L_2 з/без ранньої зупинки

11) $u(x,0) = \sin(x+1)$; $u(0,t) = t + 0{,}8415$; $u(1,t) = 4t + 0{,}9093$;

Додатково:

1. Змінити масштаби входів: [0,1]vs [−1,1]; порівняти.
2. Побудувати часові серії $u(0.25,t),u(0.5,t),u(0.75,t)$.

12) $u(x,0) = 2x(x-1)$; $u(0,t) = 4t$; $u(1,t) = 5t$;

Додатково:

1. Дослідити `tanh` vs Swish — збіжність/точність.
2. Дослідити вплив $N_f \in \{5\text{k},10\text{k},20\text{k}\}$ на точність.
3. Ввести мале $s(x,t) = 0.5\,e^{-80(x-0.5)^2}\mathbf{1}_{t>0.4}$; показати вплив на профілі.

13) $u(x,0) = 0{,}4 + \sin x$; $u(0,t) = 0{,}4 + 2t$; $u(1,t) = 1{,}2415$;

Додатково:

1. Порівняти Adam проти Adam→L-BFGS.
2. Дослідити вплив $\lambda_b \in \{5,10,20\}\}$ на порушення ГУ.
3. Порівняти L_2 з/без ранньої зупинки

14) $u(x,0) = \cos(2x + 0{,}5)$; $u(0,t) = 0{,}8776 - t$; $u(1,t) = 6t - 0{,}8011$;

Додатково:

1. Збільшити глибину з 6×128 до 8×200 — вплив на збіжність.
2. Додати слабкий Робін на $x = 1$: $-u_x = \mathrm{Bi}(u - u_\infty)$ з $\mathrm{Bi} = 1$, $u_\infty = u(1,t)$ базового кейсу; оцінити зміну потоку.

15) $u(x,0) = (x-0{,}2)(x+1)$; $u(0,t) = 6t - 0{,}2$; $u(1,t) = 1{,}6000$;

Додатково:

1. Перевірити тепловий баланс на [0,1] (для s≡0 баланс зводиться до потоку на межах).
2. Дослідити вплив $N_f \in \{5\text{k},10\text{k},20\text{k}\}$ на точність.

16) $u(x,0) = x(2x + 0{,}5)$; $u(0,t) = 2t$; $u(1,t) = 2t + 2{,}5$;

Додатково:

1. Ввести мале $s(x,t) = 0.5\,e^{-80(x-0.5)^2}\mathbf{1}_{t>0.4}$; показати вплив на профілі.
2. Порівняти рівномірний семплінг vs LHS.

17) $u(x,0) = \sin(2x + 0{,}3)$; $u(0,t) = 3t + 0{,}2955$; $u(1,t) = 4t + 0{,}7457$;

Додатково:

1. Дослідити `tanh` vs Swish — збіжність/точність.
2. Побудувати часові серії $u(0.3,t),u(0.6,t),u(0.9,t)$.
3. Змінити масштаби входів: [0,1]vs [−1,1]; порівняти.

18) $u(x,0) = \cos(x + 0{,}4)$; $u(0,t) = 6t + 0{,}9211$; $u(1,t) = 2t + 0{,}16997$;

Додатково:

1. Зробити curriculum у часі (спершу t≤0.4), порівняти точність.

2. `tanh` vs `sin` (SIREN) — порівняти L_2.

19) $u(x,0) = 2 - x(1-x)$; $u(0,t) = 2(1+t)$; $u(1,t) = 2$;

Додатково:

1. Дослідити `tanh` vs Swish — збіжність/точність.
2. Побудувати часові серії $u(0.3,t), u(0.6,t), u(0.9,t)$.

20) $u(x,0) = (x^2 + 1{,}6)\sin \pi x$; $u(0,t) = 8t$; $u(1,t) = 15t$.

Додатково:

1. Порівняти рівномірний семплінг vs LHS.
2. Порівняти Adam проти Adam→L-BFGS.
3. Дослідити вплив $\lambda_b \in \{5,10,20\}\}$ на порушення ГУ.

Зміст звіту

1. Титульний аркуш із зазначенням назви лабораторної роботи, номера варіанта, прізвища студента та навчальної групи.
2. Програмні коди, модифіковані відповідно до параметрів варіанта індивідуального завдання.
3. Результати виконання програм, що включають PINN (теплопровідність):

- Формальна постановка задачі (безрозмірна);
- Таблиця гіперпараметрів моделі і навчання;
- Криві збіжності складників втрат;
- Основні метрики точності;
- Візуалізації:
  карти поля $u(x,t)$, карти похибок, профілі $u(\cdot,t_k)$ для 2–4 моментів часу, теплові потоки на межах.
  - Індивідуальне завдання (специфічні артефакти):
- Дослідження впливу ваг втрат.
- Порівняння активацій `tanh` vs `sin` (SIREN): точність, швидкість збіжності.
- Ефект семплінгу: Uniform vs LHS vs Importance; карта щільності точок та її вплив на похибку.
- Вплив джерела.

4. Аналітичні результати досліджень та висновки, що містять Вплив гіперпараметрів:

- баланс між виконанням PDE та умов, поява/зникнення «витоків» на межі, чутливість збіжності.
- Архітектура (глибина/ширина) і активація: здатність відтворювати круті градієнти/осциляції, стабільність градієнтів (роль ініціалізації).
- Семплінг і resampling: стабільність і узагальнювальна здатність; вплив підвищеної щільності в зонах великих градієнтів.
- Оптимізація: критерії зупинки; вплив `learning rate`.